

ЛАБОРАТОРНА РОБОТА №1.

ОСНОВНІ БІБЛІОТЕКИ. ЛІНІЙНІ МОДЕЛІ НАВЧАННЯ.

Мета: ознайомитись з сервісом Colab; ознайомитися з базовими бібліотеками для машинного навчання; тренування неглибокі лінійні моделі машинного навчання на базі простих регресій: лінійна, логістична, поліноміальна.

Хід роботи:

Завдання 0. Налаштування середовища

```
[9] 0 сек. import sys
import os
import random
import numpy as np
import torch
import pandas as pd
import matplotlib.pyplot as plt
```

Рисунок 1 – Результат виконання програми

✓ 0.1. Перевіряємо чи ми точно в Colab

```
[10] 0 сек. IN_COLAB = "google.colab" in sys.modules
print("Running in Colab:", IN_COLAB)

assert IN_COLAB, f"ця лаба має вокинутися в Google Colab (https://colab.research.google.com/)"
```

... Running in Colab: True

Рисунок 2 – Результат виконання програми

✓ 0.2. Фіксуємо всі джерела випадковості.

На GPU повна детермінованість трохи сповільнює тренування (`torch.backends.cudnn.deterministic = True` вимикає частину швидких алгоритмів) — тут це не критично, адже дані й модель дрібні.

```
[11] 0 сек. SEED = 42

def set_seed(seed: int = SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

Рисунок 3 – Результат виконання програми

					ДУ «Житомирська політехніка».26.121.20.001 – Лр1							
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з лабораторної роботи			Лім.	Арк.	Аркушів		
Розроб.	Хоменко Н.В.									1	20	
Перевір.	Фант М.О.							ФІКТ Гр. ІПЗ-23-4				
Керівник												
Н. контр.												
Зав. каф.	Вакалюк Т.А.											

Вправа 1. Скористайся вбудованим модулем `random`. Спробуй `random.randint(1, 100)` і `random.shuffle()` на списку — випадковість буває не лише в числі, а й у перестановці. Згенеруй обома способами двічі ДО виклику `set_seed()` — переконайся, що результати різні. Потім ще двічі ПІСЛЯ з викликом `set_seed()` після кожної генерації і потім ще двічі без проміжного виклику `set_seed()`. Після клітинки з кодом створи клітинку з маркдауном і поясни що відбувається після `set_seed()` і як це проявляється в результатах виконання коду.

Лістинг програми:

```
numbers = list(range(10)) # список для shuffle

# TODO: двічі ДО set_seed() -- наприклад random.randint(1, 100) і random.shuffle(numbers)
print("--- двічі ДО set_seed() ---")

print("Спроба 1:")
print("randint:", random.randint(1, 100))
nums_copy1 = numbers.copy()
random.shuffle(nums_copy1)
print("shuffle:", nums_copy1)

print("Спроба 2:")
print("randint:", random.randint(1, 100))
nums_copy2 = numbers.copy()
random.shuffle(nums_copy2)
print("shuffle:", nums_copy2)

set_seed()
print("Seed зафіксовано:", SEED)

# TODO: тепер двічі ПІСЛЯ set_seed() -- виклич set_seed() перед КОЖНИМ окремим генеруванням
print("\n--- двічі ПІСЛЯ set_seed() ---")

set_seed()
print("Спроба 1:")
print("randint:", random.randint(1, 100))
nums_copy3 = numbers.copy()
random.shuffle(nums_copy3)
print("shuffle:", nums_copy3)

set_seed()
print("Спроба 2:")
print("randint:", random.randint(1, 100))
nums_copy4 = numbers.copy()
random.shuffle(nums_copy4)
print("shuffle:", nums_copy4)
```

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```
# TODO: тепер ще двічі без проміжних викликів set_seed()
print("\n--- двічі без проміжних викликів set_seed() ---")

print("Спроба 1:")
print("randint:", random.randint(1, 100))
nums_copy5 = numbers.copy()
random.shuffle(nums_copy5)
print("shuffle:", nums_copy5)

print("Спроба 2:")
print("randint:", random.randint(1, 100))
nums_copy6 = numbers.copy()
random.shuffle(nums_copy6)
print("shuffle:", nums_copy6)
```

Результат виконання програми:

```
... --- двічі ДО set_seed() ---
Спроба 1:
randint: 44
shuffle: [5, 7, 1, 9, 0, 8, 6, 3, 2, 4]
Спроба 2:
randint: 46
shuffle: [2, 6, 1, 7, 8, 3, 9, 0, 4, 5]
Seed зафіксовано: 42

--- двічі ПІСЛЯ set_seed() ---
Спроба 1:
randint: 82
shuffle: [7, 3, 2, 8, 5, 6, 9, 4, 0, 1]
Спроба 2:
randint: 82
shuffle: [7, 3, 2, 8, 5, 6, 9, 4, 0, 1]

--- двічі без проміжних викликів set_seed() ---
Спроба 1:
randint: 76
shuffle: [4, 3, 2, 9, 5, 1, 7, 8, 0, 6]
Спроба 2:
randint: 92
shuffle: [5, 1, 9, 0, 2, 4, 7, 3, 6, 8]
```

Рисунок 4 – Результат виконання програми

Вправа 2. Встанови CPU рантайм. Запусти комірку вище і зроби скріншот виводу. Потім у Colab зміни рантайм на GPU, перезапусти всі комірки від початку. Знову зроби скріншот тієї самої комірки. Збережи обидва скріншоти — пізніше завантажиш їх на GitHub разом зі задачею лаби.

0.3. Використовуємо лише CPU (в цій лабі)

На Colab зазвичай є безкоштовний GPU (T4), але для лінійної регресії на кількості точок CPU часто не повільніший — виграш GPU з'являє пересилання даних на пристрій і назад щопроходу. Тому нижче свідомо тренуємось на CPU.

```
[15] 0 сек.
device_available = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device = torch.device("cpu")

print("Доступний пристрій :", device_available)
print("Використовуємо      :", device)
if device_available.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(0))

assert device.type == "cpu", f"Ця лаба має виконуватися на CPU, а не на GPU ({device})"
```

... Доступний пристрій : cpu
Використовуємо : cpu

Рисунок 5 – Результат виконання програми

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

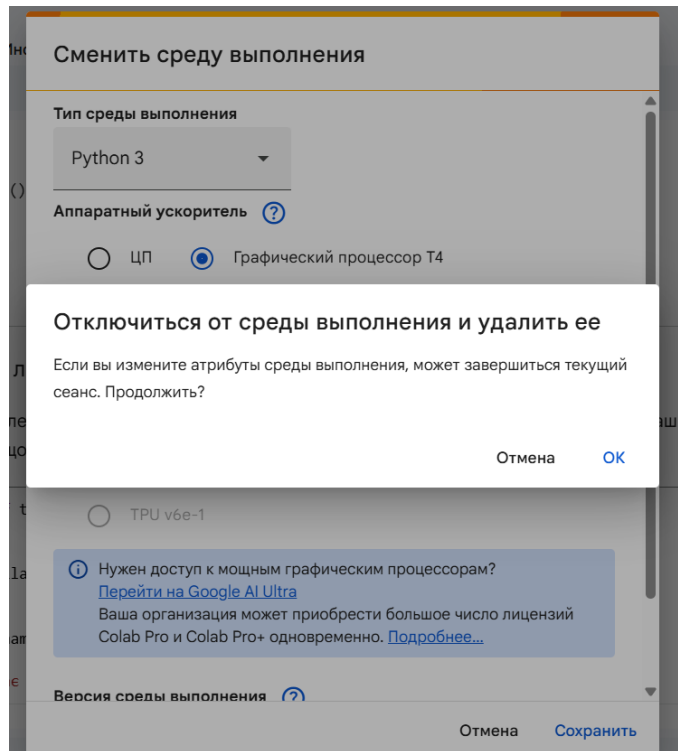


Рисунок 6 – Результат виконання програми

0.3. Використовуємо лише CPU (в цій лабі)

На Colab зазвичай є безкоштовний GPU (T4), але для лінійної регресії на кількості точок CPU часто не повільніший — виграш GPU з'їдає пересилання даних на пристрій і назад щопроходу. Тому нижче свідомо тренуємось на CPU.

```
[4] 0 сек.
device_available = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device = torch.device("cpu")

print("Доступний пристрій :", device_available)
print("Використовуємо      :", device)
if device_available.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(0))

assert device.type == "cpu", f"Ця лаба має виконуватися на CPU, а не на GPU ({device})"
```

... Доступний пристрій : cuda
Використовуємо : cpu
GPU: Tesla T4

Рисунок 7 – Результат виконання програми

0.4. Монтуюємо Google Drive

Сесія Colab обмежена в часі й може скинутись — усе в `/content` зникає разом з нею. Монтуюємо Drive, щоб чекпоінт моделі й логи TensorBoard пережили перезапуск.

```
[8] 0 сек.
from google.colab import drive
drive.mount("/content/drive")
ARTIFACT_DIR = "/content/drive/MyDrive/ai_systems/lab1"

os.makedirs(ARTIFACT_DIR, exist_ok=True)
RUNS_DIR = os.path.join(ARTIFACT_DIR, "runs")
os.makedirs(RUNS_DIR, exist_ok=True)
print("Артефакти зберігаються в:", ARTIFACT_DIR)
```

... Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Артефакти зберігаються в: /content/drive/MyDrive/ai_systems/lab1

Рисунок 8 – Результат виконання програми

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

Завдання 1. Ознайомлення з бібліотеками

Кілька дуже простих вправ на NumPy, pandas, PyTorch і Matplotlib — щоб мати спільну базу перед тим, як усі чотири підуть у хід одночасно.

1.1. NumPy

Вправа 3. Створи масив форми 3×4 , заповнений нулями (`np.zeros`), і ще один такої самої форми, заповнений числом 7 (`np.full`). Виведи форму (`.shape`) і тип даних (`.dtype`) обох.

Лістинг програми:

```
# TODO: zeros_arr = np.zeros((3, 4))
zeros_arr = np.zeros((3, 4))

# TODO: sevens_arr = np.full((3, 4), 7)
sevens_arr = np.full((3, 4), 7)

print(zeros_arr, zeros_arr.shape if zeros_arr is not None else None, zeros_arr.dtype if
zeros_arr is not None else None)
print(sevens_arr, sevens_arr.shape if sevens_arr is not None else None, sevens_arr.dtype if
sevens_arr is not None else None)
```

Результат виконання програми:

```
... [[0. 0. 0. 0.]
      [0. 0. 0. 0.]
      [0. 0. 0. 0.]] (3, 4) float64
[[7 7 7 7]
 [7 7 7 7]
 [7 7 7 7]] (3, 4) int64
```

Рисунок 9 – Результат виконання програми

Вправа 4. Створи масив від 0 до 20 із кроком 2 через `np.arange`, і масив із 5 рівномірно розподілених точок від 0 до 1 через `np.linspace`. Виведи обидва.

Лістинг програми:

```
# TODO: arange_arr = np.arange(0, 20, 2)
arange_arr = np.arange(0, 20, 2)

# TODO: linspace_arr = np.linspace(0, 1, 5)
linspace_arr = np.linspace(0, 1, 5)

print(arange_arr)
print(linspace_arr)
```

Результат виконання програми:

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```
... [ 0  2  4  6  8 10 12 14 16 18]
     [0.  0.25 0.5  0.75 1.  ]
```

Рисунок 10 – Результат виконання програми

Вправа 5. Створи одиничну матрицю 4×4 (np.eye) і випадковий масив цілих чисел форми 3×3 у діапазоні [0, 10) (np.random.randint). Виведи .ndim обох масивів.

Лістинг програми:

```
# TODO: identity = np.eye(4)
identity = np.eye(4)
# TODO: random_ints = np.random.randint(0, 10, size=(3, 3))
random_ints = np.random.randint(0, 10, size=(3, 3))

print(identity)
print(random_ints)

print(identity.ndim)
print(random_ints.ndim)
```

Результат виконання програми:

```
... [[1. 0. 0. 0.]
      [0. 1. 0. 0.]
      [0. 0. 1. 0.]
      [0. 0. 0. 1.]]
     [[4 3 7]
      [7 2 5]
      [4 1 7]]
     2
     2
```

Рисунок 11 – Результат виконання програми

Вправа 6. Маєш масив оцінок студентів. Порахуй середнє, стандартне відхилення і z-normalized версію масиву — без циклів.

Лістинг програми:

```
scores = np.array([78, 85, 92, 67, 90, 74, 88, 95, 60, 82])

# TODO: порахувати mean, std і z_scores = (scores - mean) / std
mean = scores.mean()
std = scores.std()
z_scores = (scores - mean) / std

print("Середнє:", mean, " Std:", std)
print("Z-scores:", z_scores)
```

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

Результат виконання програми:

```
*** Середнє: 81.1 Std: 10.76522178127325
Z-scores: [-0.28796434  0.36227772  1.01251978 -1.30977329  0.82673633 -0.65953123
 0.64095289  1.29119495 -1.96001536  0.08360255]
```

Рисунок 12 – Результат виконання програми

Вправа 7. Маєш 1D масив із 12 чисел. Перетвори його на матрицю 3×4 і порахуй суму кожного рядка та кожного стовпця.

Лістинг програми:

```
arr = np.arange(1, 13)

# TODO: перетворити arr на матрицю 3x4 (reshape), порахувати суми по рядках
# (axis=1) і по стовпцях (axis=0)
matrix = arr.reshape(3,4)
row_sums = matrix.sum(axis=1)
col_sums = matrix.sum(axis=0)

print(matrix)
print("Сума по рядках: ", row_sums)
print("Сума по стовпцях:", col_sums)
```

Результат виконання програми:

```
✓ *** [[ 1  2  3  4]
       [ 5  6  7  8]
       [ 9 10 11 12]]
      Сума по рядках: [10 26 42]
      Сума по стовпцях: [15 18 21 24]
```

Рисунок 13 – Результат виконання програми

1.2. pandas

Вправа 8. Створи таблицю студентів з оцінками. Виведи тільки тих, хто набрав більше 80 балів, відсортованих за спаданням оцінки.

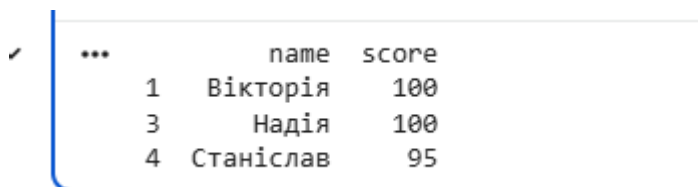
Лістинг програми:

```
df = pd.DataFrame({
    "name": ["Артем", "Вікторія", "Денис", "Надія", "Станіслав"],
    "score": [80, 100, 80, 100, 95]
}) # додай тут дані для DataFrame

# TODO: відфільтрувати score > 80 і відсортувати за score за спаданням
top = df[df["score"] > 80].sort_values("score", ascending=False)
print(top)
```

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

Результат виконання програми:



...	name	score
1	Вікторія	100
3	Надія	100
4	Станіслав	95

Рисунок 14 – Результат виконання програми

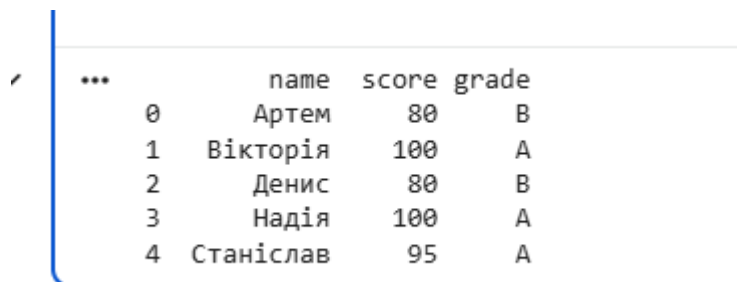
Вправа 9. Додай колонку grade: А якщо score ≥ 90 , В якщо score ≥ 75 , інакше С.

Лістинг програми:

```
# TODO: додати колонку "grade"
# умови: score >= 90 -> "А", score >= 75 -> "В", інакше -> "С"
def get_grade(score):
    if score >= 90: return "А"
    elif score >= 75: return "В"
    else: return "С"

df["grade"] = df["score"].apply(get_grade)
print(df)
```

Результат виконання програми:



...	name	score	grade
0	Артем	80	В
1	Вікторія	100	А
2	Денис	80	В
3	Надія	100	А
4	Станіслав	95	А

Рисунок 15 – Результат виконання програми

1.3. PyTorch

Вправа 10. Створи тензор форми 2×3 , заповнений нулями (torch.zeros), і ще один такої самої форми, заповнений числом 7 (torch.full). Виведи форму (.shape) і тип даних (.dtype) обох.

Лістинг програми:

```
# TODO: zeros_t = torch.zeros((2, 3))
# TODO: sevens_t = torch.full((2, 3), 7)
zeros_t = torch.zeros((2, 3))
sevens_t = torch.full((2, 3), 7)

print(zeros_t)
print(sevens_t)
```

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				8
Змн.	Арк.	№ докум.	Підпис	Дата		

```
print(zeros_t.shape, zeros_t.dtype)
print(sevens_t.shape, sevens_t.dtype)
```

Результат виконання програми:

```
... tensor([[0., 0., 0.],
           [0., 0., 0.]])
tensor([[7, 7, 7],
        [7, 7, 7]])
torch.Size([2, 3]) torch.float32
torch.Size([2, 3]) torch.int64
```

Рисунок 16 – Результат виконання програми

Вправа 11. Створи тензор від 0 до 20 із кроком 2 через `torch.arange`, і тензор із 5 рівномірно розподілених точок від 0 до 1 через `torch.linspace`.

Лістинг програми:

```
# TODO: arange_t = torch.arange(0, 20, 2)
# TODO: linspace_t = torch.linspace(0, 1, 5)
arange_t = torch.arange(0, 20, 2)
linspace_t = torch.linspace(0, 1, 5)

print(arange_t)
print(linspace_t)
```

Результат виконання програми:

```
... tensor([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

Рисунок 17 – Результат виконання програми

Вправа 12. Створи тензор напряду зі списку — `torch.tensor([[1, 2], [3, 4]])` — і випадковий тензор форми 3×3 через `torch.rand`. Виведи `.shape`, `.dtype` і `.ndim` обох.

Лістинг програми:

```
# TODO: from_list = torch.tensor([[1, 2], [3, 4]])
# TODO: random_t = torch.rand(3, 3)
from_list = torch.tensor([[1, 2], [3, 4]])
random_t = torch.rand(3, 3)

print(from_list)
print(random_t)

print(from_list.shape, from_list.dtype, from_list.ndim)
print(random_t.shape, random_t.dtype, random_t.ndim)
```

Результат виконання програми:

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

```

... tensor([[1, 2],
           [3, 4]])
      tensor([[0.8823, 0.9150, 0.3829],
           [0.9593, 0.3904, 0.6009],
           [0.2566, 0.7936, 0.9408]])
      torch.Size([2, 2]) torch.int64 2
      torch.Size([3, 3]) torch.float32 2

```

Рисунок 18 – Результат виконання програми

Вправа 13. Створи тензор $x = 5.0$ з `requires_grad=True`, порахуй $y = x^{**2}$ і перевір автоматичний градієнт проти аналітичного ($dy/dx=2x$).

Лістинг програми:

```

n = 5.0

# TODO: x = torch.tensor(n, requires_grad=True), y = x ** 2, y.backward()
x = torch.tensor(n, requires_grad=True)
y = x ** 2
y.backward()

# надрукувати x.grad.item() і порівняти з 2 * n
print("Автоматичний градієнт:", x.grad.item())
print("Аналітичний градієнт (2*n):", 2 * n)

```

Результат виконання програми:

```

... Автоматичний градієнт: 10.0
    Аналітичний градієнт (2*n): 10.0

```

Рисунок 19 – Результат виконання програми

Вправа 14. Знайди x , який мінімізує $(x-3)^2+5$ — не аналітично, а градієнтним спуском: x як параметр з `requires_grad=True`, без жодної моделі.

Лістинг програми:

```

x = torch.tensor(0.0, requires_grad=True)
lr = 0.1

# TODO: цикл градієнтного спуску (20 кроків):
#   loss = (x - 3) ** 2 + 5
#   loss.backward()
#   під torch.no_grad(): x -= lr * x.grad
#   x.grad.zero_()
for step in range(20):
    loss = (x - 3) ** 2 + 5
    loss.backward()

```

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

```

with torch.no_grad():
    x -= lr * x.grad

x.grad.zero_()

print(f"x = {x.item():.4f} (мінімум у x=3)")

```

Результат виконання програми:

```

... x = 2.9654 (мінімум у x=3)

```

Рисунок 20 – Результат виконання програми

1.4. Matplotlib

Вправа 15. Побудуй графік функції $y=\sin(x)$ на проміжку $[0, 2\pi]$.

Лістинг програми:

```

# TODO: xs = np.linspace(0, 2*np.pi, 100), plt.plot(xs, np.sin(xs)),
plt.show()
xs = np.linspace(0, 2*np.pi, 100)

plt.plot(xs, np.sin(xs))
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("y = sin(x)")
plt.grid(alpha=0.3)
plt.show()

```

Результат виконання програми:

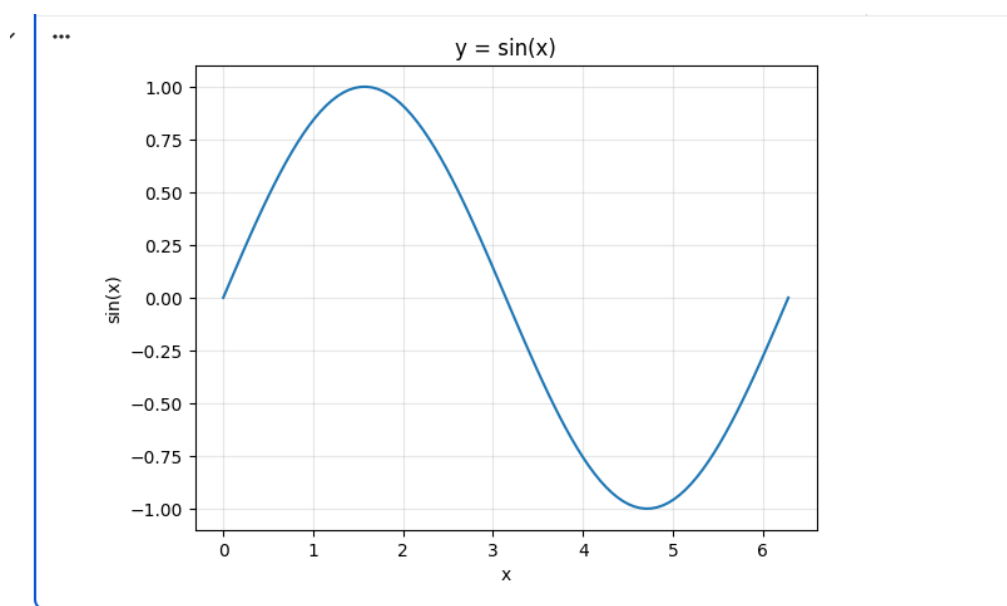


Рисунок 21 – Результат виконання програми

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк. 11
		Фант М.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		

Вправа 16. На одному графіку познач дві групи точок різними кольорами і додай легенду.

Лістинг програми:

```
group_a = np.random.randn(30) + 2
group_b = np.random.randn(30) - 2

# TODO: scatter обох груп різними кольорами (color="C0"/"C1", label=...),
plt.scatter(range(len(group_a)), group_a, color="C0", label="Група А")
plt.scatter(range(len(group_b)), group_b, color="C1", label="Група В")
plt.xlabel("Індекс точки")
plt.ylabel("Значення")
plt.title("Дві групи точок")

# додати plt.legend(), plt.show()
plt.legend()
plt.show()
```

Результат виконання програми:

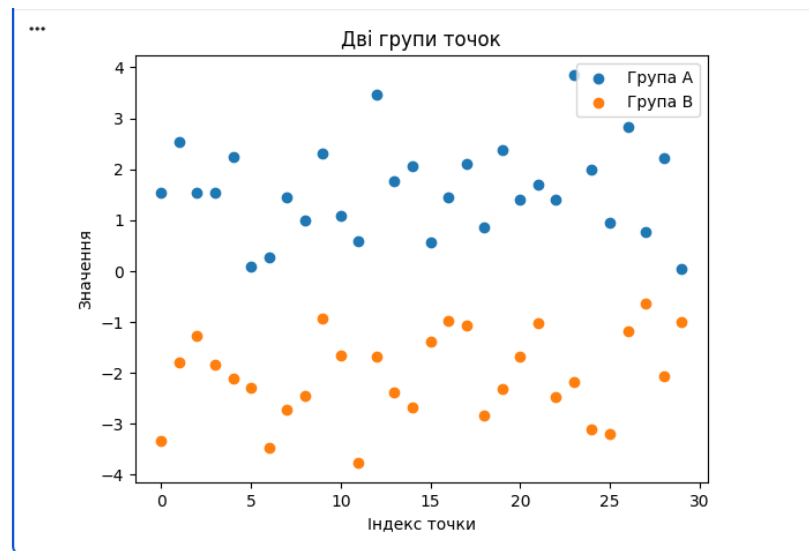


Рисунок 22 – Результат виконання програми

Завдання 1. Лінійна регресія

Тренуємо базовий пайплайн на простій лінійній регресії з одним параметром.

$$y = w \cdot x + b + \epsilon, \epsilon \sim N(0, 1)$$

```
[30]
0 сек.
TASK1_TRUE_W = 2.0
TASK1_TRUE_B = 1.0
TASK1_N = 300

x1 = torch.empty(TASK1_N, 1).uniform_(-5, 5)
noise = 1.0 * torch.randn(TASK1_N, 1)
y1 = TASK1_TRUE_W * x1 + TASK1_TRUE_B + noise
```

Рисунок 23 – Результат виконання програми

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				12
Змн.	Арк.	№ докум.	Підпис	Дата		

Вправа 17. Перш ніж будувати модель — подивись на самі дані.

Лістинг програми:

```
# TODO: Точкова діаграма (scatter plot) з вісями x1 та y1, підпис осей
(xlabel/ylabel), заголовок
plt.scatter(x1.numpy(), y1.numpy(), s=8, alpha=0.5)
plt.xlabel("x1")
plt.ylabel("y1")

# "Сирі дані (до тренування)"
plt.title("Сирі дані (до тренування)")
plt.grid(alpha=0.3)
plt.show()
```

Результат виконання програми:

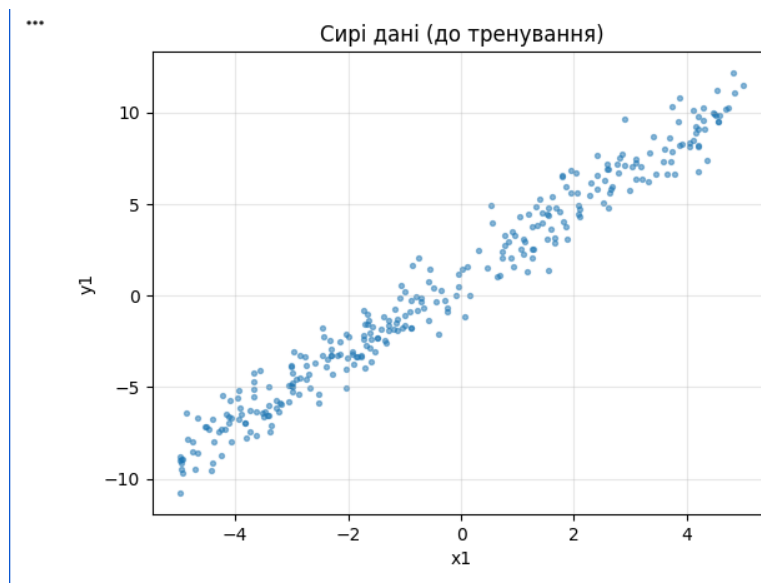


Рисунок 24 – Результат виконання програми

Вправа 18. Розбий дані на train/val/test (70/15/15). Перемішай дані перед зрізом, інакше спліт вийде не випадковим.

Лістинг програми:

```
# TODO: перемішати x1/y1 через torch.randperm(TASK1_N), розбити на
# train/val/test у пропорції 70/15/15
# x_train1, y_train1 = ...
# x_val1, y_val1 = ...
# x_test1, y_test1 = ...
perm = torch.randperm(TASK1_N)

n_train = int(0.7 * TASK1_N)
n_val = int(0.15 * TASK1_N)

train_idx = perm[:n_train]
val_idx = perm[n_train:n_train + n_val]
test_idx = perm[n_train + n_val:]
```

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				13
Змн.	Арк.	№ докум.	Підпис	Дата		

```

x_train1, y_train1 = x1[train_idx], y1[train_idx]
x_val1, y_val1 = x1[val_idx], y1[val_idx]
x_test1, y_test1 = x1[test_idx], y1[test_idx]

print(f"train/val/test: {len(x_train1)}/{len(x_val1)}/{len(x_test1)}")

```

Результат виконання програми:

```
... train/val/test: 210/45/45
```

Рисунок 25 – Результат виконання програми

Вправа 19. Нижче — готовий цикл навчання, кожен крок підписаний окремо. Просто запусти обидві комірки й подивись, як модель наближається до true за 100 епох (знімки кожні 30 епох).

```

[36] 7 ✓ СЕК. task1_model = torch.nn.Linear(in_features=1, out_features=1)
optimizer = torch.optim.SGD(task1_model.parameters(), lr=0.05)
loss_fn = torch.nn.MSELoss()

x_line = torch.linspace(-5, 5, 100).unsqueeze(1) # сітка для візуалізації лінії
snapshots = [] # (epoch, w, b, y_line_pred) кожні 30 епох

for epoch in range(100):
    # обнулити градієнти з попереднього кроку
    optimizer.zero_grad()
    # forward -- прогноз моделі на поточних w, b
    y_pred = task1_model(x_train1)
    # loss -- наскільки прогноз відрізняється від y_train1
    loss = loss_fn(y_pred, y_train1)
    # backward -- градієнти loss за w i b
    loss.backward()
    # крок оптимізатора -- оновити w i b
    optimizer.step()

    if epoch % 30 == 0 or epoch == 99:
        with torch.no_grad():
            y_line_pred = task1_model(x_line)
            snapshots.append((epoch, task1_model.weight.item(), task1_model.bias.item(), y_line_pred))
        print(f"epoch {epoch:4d} loss={loss.item():.6f} "
              f"w={task1_model.weight.item():.4f} (true {TASK1_TRUE_W}) "
              f"b={task1_model.bias.item():.4f} (true {TASK1_TRUE_B})")

```

```

... epoch    0 loss=16.105141 w=1.7198 (true 2.0) b=0.9313 (true 1.0)
epoch    30 loss=0.928843 w=1.9580 (true 2.0) b=0.9466 (true 1.0)
epoch    60 loss=0.928841 w=1.9580 (true 2.0) b=0.9477 (true 1.0)
epoch    90 loss=0.928841 w=1.9580 (true 2.0) b=0.9477 (true 1.0)
epoch   99 loss=0.928841 w=1.9580 (true 2.0) b=0.9477 (true 1.0)

```

Рисунок 26 – Результат виконання програми

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				14
Змн.	Арк.	№ докум.	Підпис	Дата		

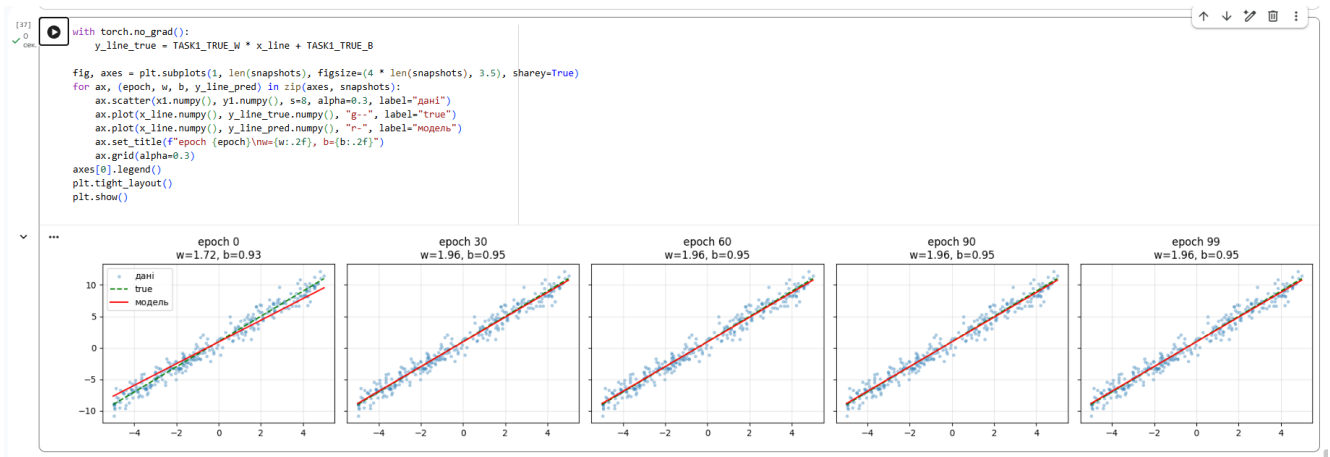


Рисунок 27 – Результат виконання програми

Вправа 20. Придумай власний напів-реальний сценарій лінійної залежності (наприклад: кількість з'їдених пачок чіпсів на тиждень $\rightarrow$ індекс маси тіла; години сну $\rightarrow$ продуктивність; щось своє). У markdown-комірці перед кодом опиши сценарій: яку залежність ти закладаєш (w , b) і чому саме такі значення видались тобі правдоподібними. Згенеруй дані з шумом, збережи їх у CSV файл (за зразком відповідної частини лекції) і натренуй лінійну регресію за тим самим патерном — спліт, цикл навчання, перевірка на test. Організуй код тренування у логічні перевикористовувані функції.

Власний сценарій: кількість пар на тиждень $\rightarrow$ вільний час

Сценарій. Досліджуємо залежність між кількістю пар у студента за тиждень і кількістю годин вільного часу, який залишається на тиждень. Логічно припустити, що чим більше пар відвідує студент, тим менше вільного часу в нього залишається — не лише через саму присутність на парах, а й через час на підготовку до них, дорогу до університету тощо.

Модель залежності: $\text{вільний_час} = w \cdot \text{кількість_пар} + b + \epsilon$

Обрані параметри:

- $w = -1.5$ — кожна додаткова пара на тиждень "забирає" в середньому 1.5 години вільного часу. Це число враховує не тільки саму пару (зазвичай ~ 1.2 год), а й непрямі витрати часу навколо неї (підготовка, дорога), тому коефіцієнт трохи вищий за "голу" тривалість однієї пари.

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				15
Змн.	Арк.	№ докум.	Підпис	Дата		

- $b=40$ — базовий рівень вільного часу на тиждень при нульовій кількості пар. Тиждень має 168 годин; віднявши приблизний час на сон (56 год), побут, харчування та інші фіксовані справи (70 год), залишається орієнтовно 40 годин "гнучкого" часу, який студент міг би витратити на навчання чи відпочинок.
- Діапазон x (кількість пар на тиждень): від 0 до 30 — реалістичний діапазон для студентського навантаження.
- Шум $\varepsilon \sim N(0,3)$ — стандартне відхилення 3 години відображає те, що реальний вільний час залежить не лише від кількості пар, а й від інших факторів (робота, хобі, індивідуальний темп виконання завдань), які модель напряду не враховує.

Лістинг програми:

```
TASK2_TRUE_W = -1.5
TASK2_TRUE_B = 40.0
TASK2_N = 300

set_seed()

x2 = torch.empty(TASK2_N, 1).uniform_(0, 30) # кількість пар на тиждень
noise2 = 3.0 * torch.randn(TASK2_N, 1) # шум
y2 = TASK2_TRUE_W * x2 + TASK2_TRUE_B + noise2 # вільний час (год)

# зберігаємо у CSV
df2 = pd.DataFrame({
    "pairs_per_week": x2.squeeze().numpy(),
    "free_time_hours": y2.squeeze().numpy()
})
csv_path = os.path.join(ARTIFACT_DIR, "dataset.csv")
df2.to_csv(csv_path, index=False)
print("Дані збережено в:", csv_path)
print(df2.head())
```

Результат виконання програми:

```
✓ ... Дані збережено в: /content/drive/MyDrive/ai_systems/lab1/dataset.csv
   pairs_per_week  free_time_hours
0         26.468079         -1.373683
1         27.450119         -4.080220
2         11.485912         25.384970
3         28.779169         -3.455677
4         11.713447         23.641239
```

Рисунок 28 – Результат виконання програми

Лістинг програми:

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				16
Змн.	Арк.	№ докум.	Підпис	Дата		

```
[46] def split_data(x, y, n_total, train_frac=0.7, val_frac=0.15, seed=None):
    """Перемішує та розбиває дані на train/val/test."""
    if seed is not None:
        set_seed(seed)
    perm = torch.randperm(n_total)
    n_train = int(train_frac * n_total)
    n_val = int(val_frac * n_total)

    train_idx = perm[:n_train]
    val_idx = perm[n_train:n_train + n_val]
    test_idx = perm[n_train + n_val:]

    return (x[train_idx], y[train_idx],
            x[val_idx], y[val_idx],
            x[test_idx], y[test_idx])

def train_linear_model(x_train, y_train, n_epochs=100, lr=0.001):
    """Тренує лінійну регресію методом Adam, повертає модель і історію loss."""
    model = torch.nn.Linear(in_features=1, out_features=1)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = torch.nn.MSELoss()

    history = []
    for epoch in range(n_epochs):
        optimizer.zero_grad()
        y_pred = model(x_train)
        loss = loss_fn(y_pred, y_train)
        loss.backward()
        optimizer.step()
        history.append(loss.item())

        if epoch % 30 == 0 or epoch == n_epochs - 1:
            print(f"epoch {epoch:4d} loss={loss.item():.4f} "
                  f"w={model.weight.item():.4f} b={model.bias.item():.4f}")

    return model, history

def evaluate_model(model, x, y, loss_fn=None):
    """Рахує MSE моделі на переданому наборі даних."""
    if loss_fn is None:
        loss_fn = torch.nn.MSELoss()
    with torch.no_grad():
        y_pred = model(x)
        loss = loss_fn(y_pred, y)
    return loss.item()
```

Рисунок 29 – Результат виконання програми

Лістинг програми:

```
x_train2, y_train2, x_val2, y_val2, x_test2, y_test2 = split_data(
    x2, y2, TASK2_N, seed=SEED
)
print(f"train/val/test: {len(x_train2)}/{len(x_val2)}/{len(x_test2)}")

task2_model, history2 = train_linear_model(x_train2, y_train2, n_epochs=2000,
lr=0.1)

test_loss = evaluate_model(task2_model, x_test2, y_test2)
print(f"\nTest MSE: {test_loss:.4f}")
print(f"Знайдено: w={task2_model.weight.item():.4f},
b={task2_model.bias.item():.4f}")
print(f"True: w={TASK2_TRUE_W}, b={TASK2_TRUE_B}")
```

Результат виконання програми:

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				17
Змн.	Арк.	№ докум.	Підпис	Дата		

```

epoch 720 loss=10.4004 w=-1.3860 b=38.4311
epoch 750 loss=9.9770 w=-1.3860 b=38.4311
epoch 780 loss=9.6086 w=-1.4035 b=38.7724
epoch 810 loss=9.3463 w=-1.4184 b=39.0647
epoch 840 loss=9.1615 w=-1.4312 b=39.3134
epoch 870 loss=9.0329 w=-1.4420 b=39.5238
epoch 900 loss=8.9443 w=-1.4511 b=39.7007
epoch 930 loss=8.8841 w=-1.4586 b=39.8485
epoch 960 loss=8.8436 w=-1.4649 b=39.9713
epoch 990 loss=8.8167 w=-1.4701 b=40.0728
epoch 1020 loss=8.7991 w=-1.4744 b=40.1560
epoch 1050 loss=8.7876 w=-1.4779 b=40.2238
epoch 1080 loss=8.7803 w=-1.4807 b=40.2788
epoch 1110 loss=8.7756 w=-1.4830 b=40.3231
epoch 1140 loss=8.7727 w=-1.4848 b=40.3586
epoch 1170 loss=8.7709 w=-1.4862 b=40.3868
epoch 1200 loss=8.7698 w=-1.4874 b=40.4091
epoch 1230 loss=8.7691 w=-1.4883 b=40.4266
epoch 1260 loss=8.7687 w=-1.4890 b=40.4403
epoch 1290 loss=8.7685 w=-1.4895 b=40.4508
epoch 1320 loss=8.7684 w=-1.4899 b=40.4590
epoch 1350 loss=8.7683 w=-1.4902 b=40.4652
epoch 1380 loss=8.7683 w=-1.4905 b=40.4699
epoch 1410 loss=8.7682 w=-1.4907 b=40.4735
epoch 1440 loss=8.7682 w=-1.4908 b=40.4762
epoch 1470 loss=8.7682 w=-1.4909 b=40.4781
epoch 1500 loss=8.7682 w=-1.4910 b=40.4796
epoch 1530 loss=8.7682 w=-1.4910 b=40.4807
epoch 1560 loss=8.7682 w=-1.4911 b=40.4815
epoch 1590 loss=8.7682 w=-1.4911 b=40.4820
epoch 1620 loss=8.7682 w=-1.4911 b=40.4824
epoch 1650 loss=8.7682 w=-1.4911 b=40.4827
epoch 1680 loss=8.7682 w=-1.4912 b=40.4829
epoch 1710 loss=8.7682 w=-1.4912 b=40.4830
epoch 1740 loss=8.7682 w=-1.4912 b=40.4832
epoch 1770 loss=8.7682 w=-1.4912 b=40.4832
epoch 1800 loss=8.7682 w=-1.4912 b=40.4832
epoch 1830 loss=8.7682 w=-1.4912 b=40.4832
epoch 1860 loss=8.7682 w=-1.4912 b=40.4832
epoch 1890 loss=8.7682 w=-1.4912 b=40.4832
epoch 1920 loss=8.7682 w=-1.4912 b=40.4832
epoch 1950 loss=8.7682 w=-1.4912 b=40.4832
epoch 1980 loss=8.7682 w=-1.4912 b=40.4832
epoch 1999 loss=8.7682 w=-1.4912 b=40.4832

Test MSE: 10.2096
Знайдено: w=-1.4912, b=40.4832
True: w=-1.5, b=40.0

```

Рисунок 30 – Результат виконання програми

Лістинг програми:

```

with torch.no_grad():
    x_line2 = torch.linspace(0, 30, 100).unsqueeze(1)
    y_line_true2 = TASK2_TRUE_W * x_line2 + TASK2_TRUE_B
    y_line_pred2 = task2_model(x_line2)

plt.scatter(x2.numpy(), y2.numpy(), s=8, alpha=0.3, label="дані")
plt.plot(x_line2.numpy(), y_line_true2.numpy(), "g--", label="true")
plt.plot(x_line2.numpy(), y_line_pred2.numpy(), "r-", label="модель")
plt.xlabel("Кількість пар на тиждень")
plt.ylabel("Вільний час (год)")

```

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				18
Змн.	Арк.	№ докум.	Підпис	Дата		

```
plt.title("Кількість пар → вільний час")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Результат виконання програми:

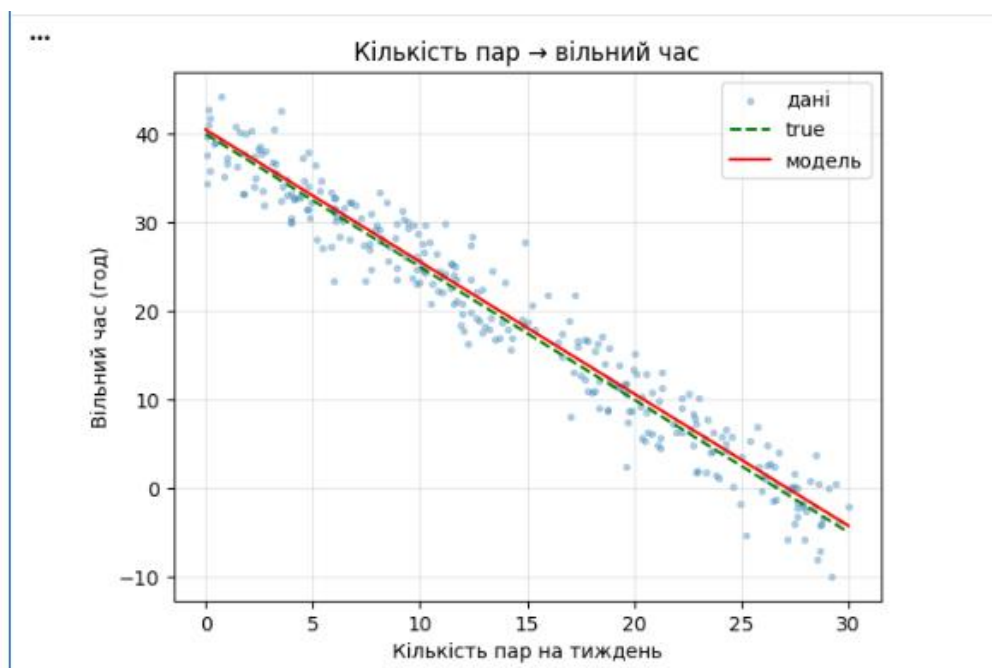


Рисунок 31 – Результат виконання програми

Вправа 21. Збережи натреновану модель на своєму гугл диску у файлі з назвою model.pt. Збережений файл потрібно буде також здати разом з лабою.

Лістинг програми:

```
model_path = os.path.join(ARTIFACT_DIR, "model.pt")
torch.save(task2_model.state_dict(), model_path)
print("Модель збережено в:", model_path)
```

Результат виконання програми:

```
... Модель збережено в: /content/drive/MyDrive/ai_systems/lab1/model.pt
```

Рисунок 32 – Результат виконання програми

Вправа 22. Встанови Gradio

```
[48] 12 !pip install -q gradio
✓ сек.

[49] 9 import gradio as gr
✓ сек.
```

Рисунок 33 – Результат виконання програми

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				19
Змн.	Арк.	№ докум.	Підпис	Дата		

Лістинг програми:

```
loaded_model = torch.nn.Linear(in_features=1, out_features=1)
loaded_model.load_state_dict(torch.load(model_path))
loaded_model.eval()

def predict_free_time(pairs_per_week):
    x_input = torch.tensor([[float(pairs_per_week)]] )
    with torch.no_grad():
        prediction = loaded_model(x_input)
    return f"Прогнозований вільний час: {prediction.item():.2f} год/тиждень"

demo = gr.Interface(
    fn=predict_free_time,
    inputs=gr.Slider(minimum=0, maximum=30, step=1, value=10, label="Кількість пар на тиждень"),
    outputs=gr.Textbox(label="Прогноз"),
    title="Прогноз вільного часу студента",
    description="Модель лінійної регресії передбачає кількість вільних годин на тиждень залежно від кількості пар."
)

demo.launch()
```

Результат виконання програми:

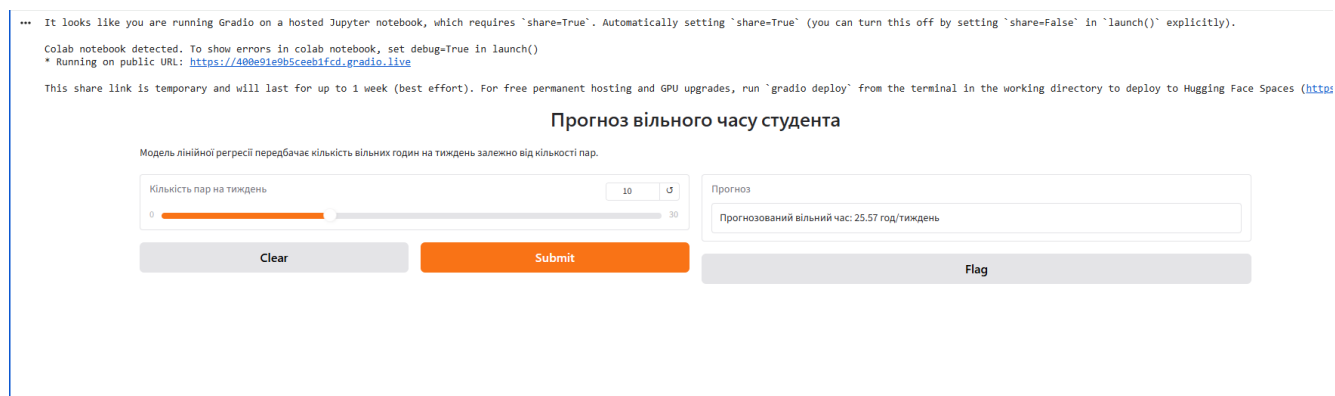


Рисунок 34 – Результат виконання програми

https://github.com/nkhmnk/ai_systems/tree/main/lab1

Висновок: ознайомилась з сервісом Colab; ознайомилась з базовими бібліотеками для машинного навчання; тренувала неглибокі лінійні моделі машинного навчання на базі простих регресій: лінійна, логістична, поліноміальна.

		Хоменко Н.В.			ДУ «Житомирська політехніка».26.121.20.001 – Лр1	Арк.
		Фант М.О.				20
Змн.	Арк.	№ докум.	Підпис	Дата		